



香港中文大學
The Chinese University of Hong Kong



Formal Languages and Automata Theory

Lecture 1

何宗易

Tsung-Yi Ho
tyho@cse.cuhk.edu.hk

Outline



Mathematical Preliminaries and Notation



Three Basic Concepts



Some Applications (omit) X

Mathematical Preliminaries

- Sets
- Functions
- Relations
- Graphs
- Proof Techniques

SETS

A set is a collection of elements

$$A = \{1, 2, 3\}$$

$$B = \{train, bus, bicycle, airplane\}$$

We write

$$1 \in A \Rightarrow 1 \text{ is an element of the set } A$$

$$ship \notin B \Rightarrow \text{ship is not an element of the set } B$$

Set Representations

$$C = \{ a, b, c, d, e, f, g, h, i, j, k \}$$

$$C = \{ a, b, \dots, k \} \longrightarrow \textit{finite set}$$

$$S = \{ 2, 4, 6, \dots \} \longrightarrow \textit{infinite set}$$

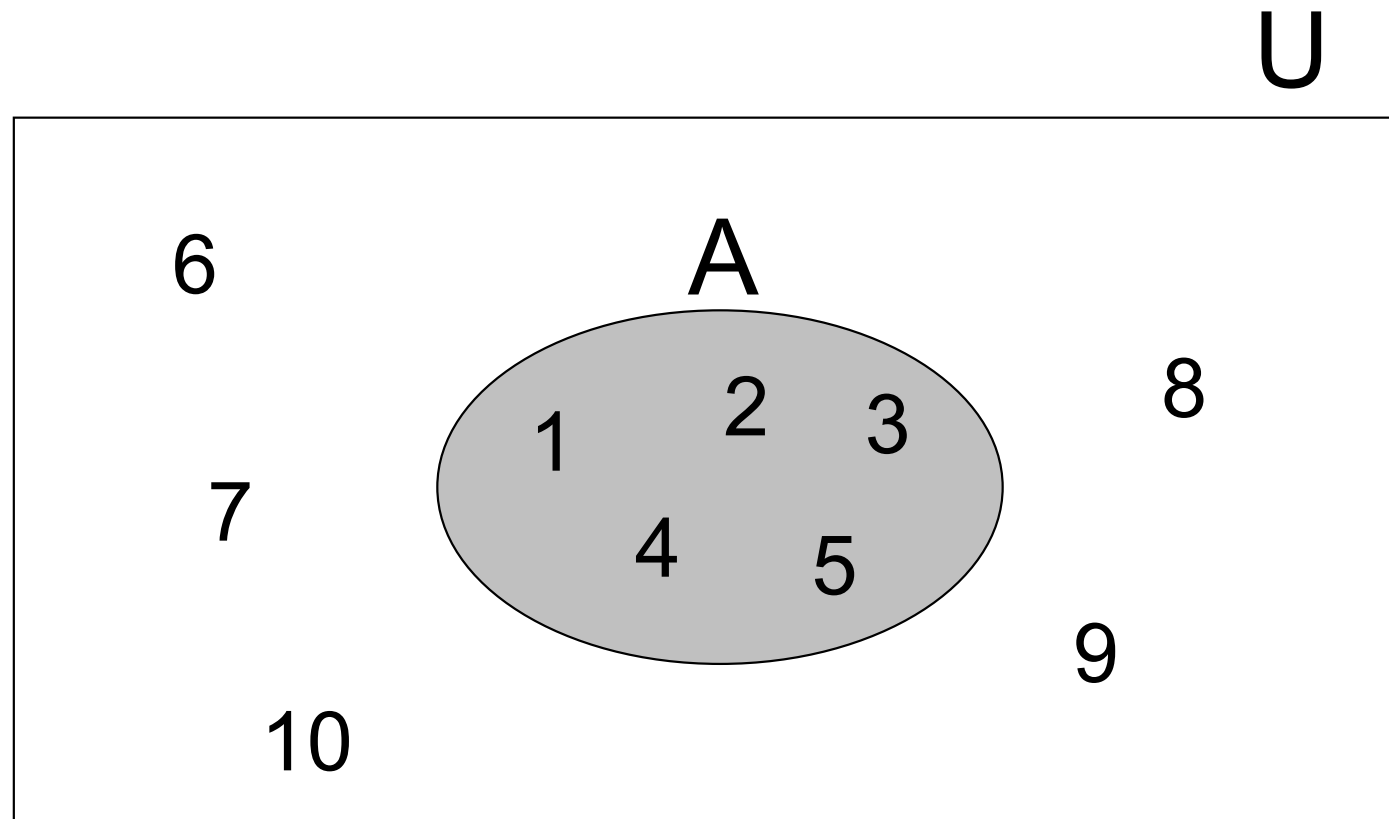
$$S = \{ j : j > 0, \text{ and } j = 2k \text{ for some } k > 0 \}$$

$\triangle$

$$S = \{ j : j \text{ is nonnegative and even} \}$$

} Explicit notation

$$A = \{ 1, 2, 3, 4, 5 \}$$



Universal Set: all possible elements

$$U = \{ 1, \dots, 10 \}$$

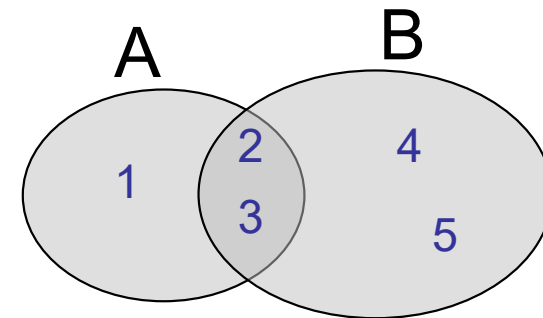
Set Operations

$$A = \{ 1, 2, 3 \}$$

$$B = \{ 2, 3, 4, 5 \}$$

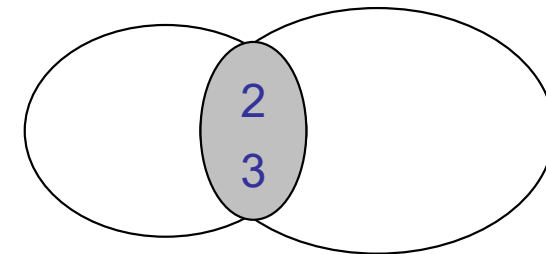
- Union (U)

$$A \cup B = \{ 1, 2, 3, 4, 5 \}$$



- Intersection ($\cap$)

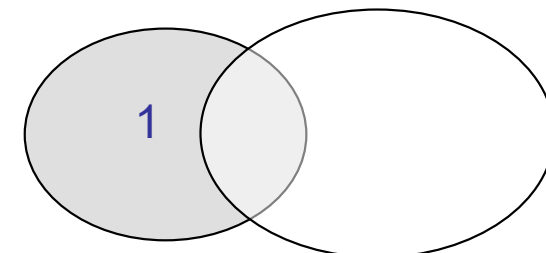
$$A \cap B = \{ 2, 3 \}$$



- Difference (-)

$$A - B = \{ 1 \}$$

$$B - A = \{ 4, 5 \}$$

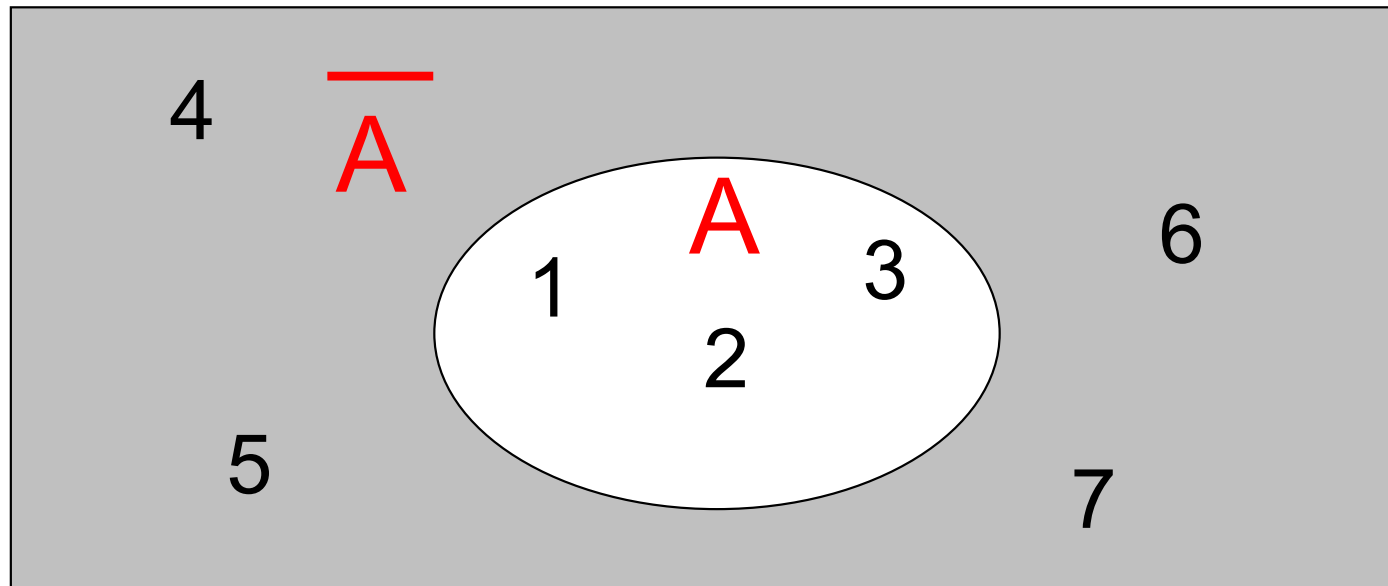


Venn diagrams

- Complement

Universal set = $\{1, \dots, 7\}$

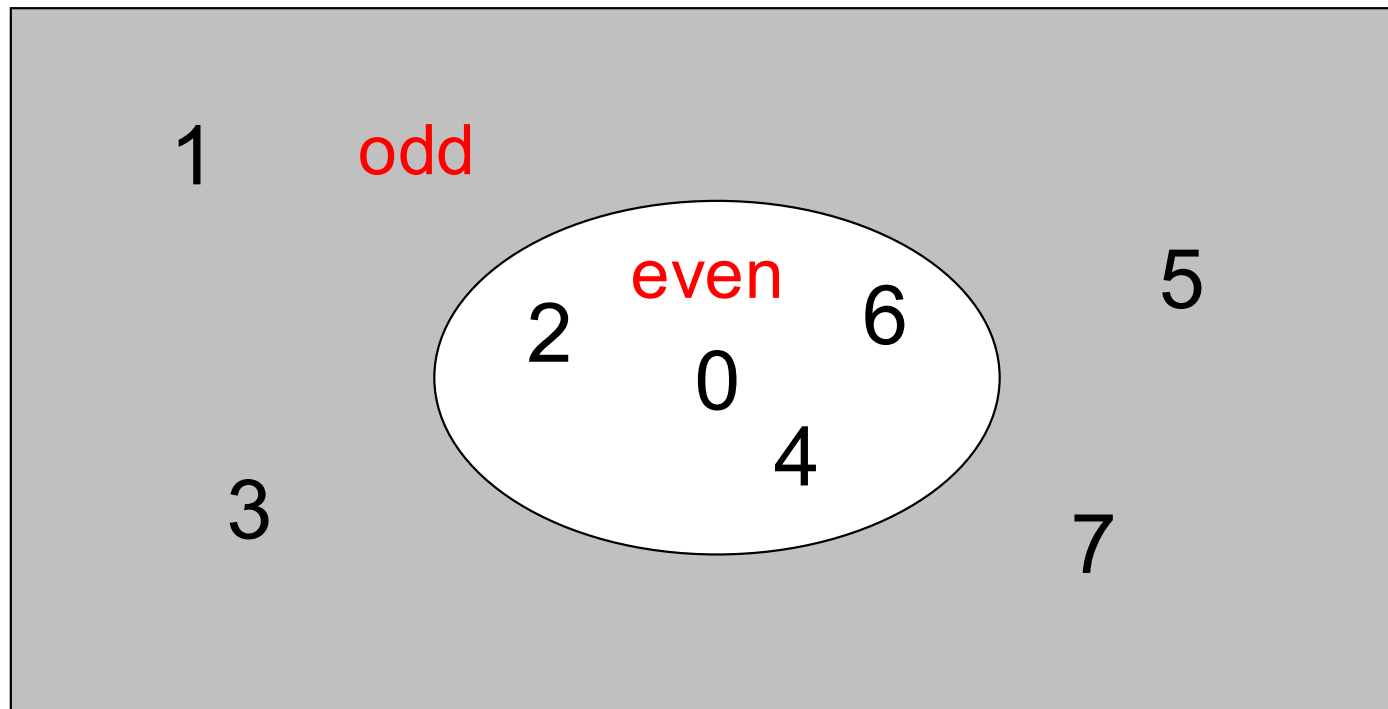
$A = \{1, 2, 3\}$ $\longrightarrow$ $\bar{A} = \{4, 5, 6, 7\}$



$$\bar{\bar{A}} = A$$

$$\{ \text{even integers} \} = \{ \text{odd integers} \}$$

Integers



DeMorgan's Laws

★ 德摩根律

$$\overline{A \cup B} = \bar{A} \cap \bar{B}$$

长变短必伴符号反。

$$\overline{A \cap B} = \bar{A} \cup \bar{B}$$

Empty, Null Set: $\emptyset$

$\emptyset = \{ \}$  The set with no elements

$$S \cup \emptyset = S$$

$$S \cap \emptyset = \emptyset$$

$$S - \emptyset = S$$

$$\emptyset - S = \emptyset$$

$\overline{\emptyset}$ = Universal Set

Subset

$$A = \{ 1, 2, 3 \}$$

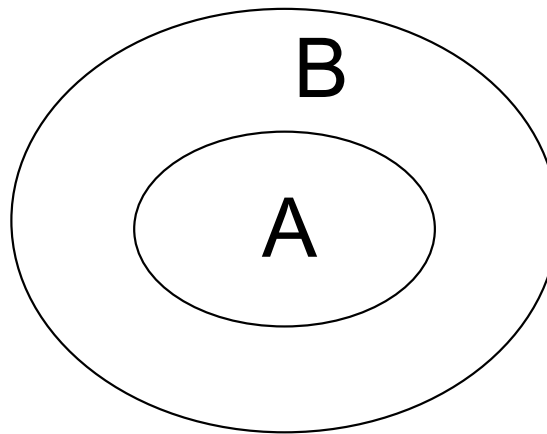
$$B = \{ 1, 2, 3, 4, 5 \}$$

$$A \subseteq B$$

Subset: If every element of A is also an element of B

Proper Subset: If $A \subseteq B$, but B contains an element not in A

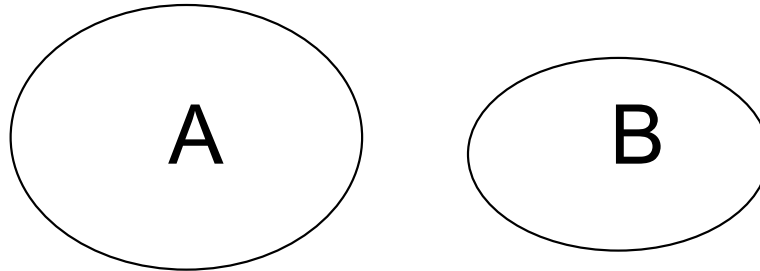
$$A \subset B$$



Disjoint Sets

$$A = \{ 1, 2, 3 \} \quad B = \{ 5, 6 \}$$

$$A \cap B = \emptyset$$



Set Cardinality

the number of elements in a set.

- For finite sets

$$A = \{ 2, 5, 7 \}$$

$$|A| = 3 \text{ (set size)}$$

Powersets *very important.*

A powerset is a set of sets *

$$S = \{ a, b, c \}$$

Powerset of S = the set of all the subsets of S *

Example 1.1

$$2^S = \{ \emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\} \}$$

*S has 3 elements
for every element, choose take or not $\Rightarrow 2$*

Observation: if S is finite, then $|2^S| = 2^{|S|}$ ($8 = 2^3$)

Cartesian Product

Example 1.2

$$A = \{ 2, 4 \}$$

$$B = \{ 2, 3, 5, 6 \}$$

$$A \times B = \{ (2, 2), (2, 3), (2, 5), (2, 6), \\ (4, 2), (4, 3), (4, 5), (4, 6) \}$$

$$|A \times B| = |A| |B|$$


Note that the **order** in which the elements of a pair are written matters



The pair (4, 2) is in $A \times B$, but (2, 4) is not

Partition

A set can be divided by separating it into a number of subsets. Suppose that $S_1, S_2, \dots, S_n$ are subsets of a given set S and that the following holds:

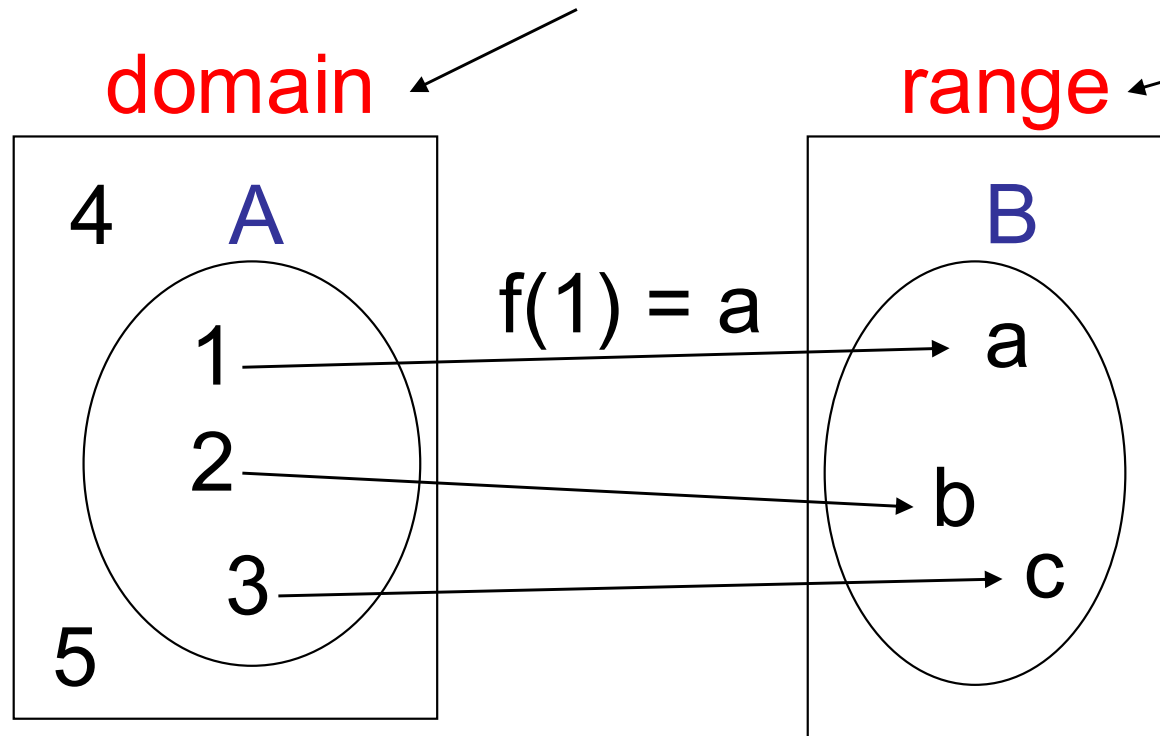
- 互斥. 非空. 子集
- ① 1. The subset $S_1, S_2, \dots, S_n$ are mutually disjoint;
 - ② 2. $S_1 \cup S_2 \cup \dots \cup S_n = S$;
 - ③ 3. None of the S_i is empty.

分割

Then $S_1, S_2, \dots, S_n$ is called a partition of S .

FUNCTIONS

Rules that assign to elements of one set a unique element of another set



$$f : A \rightarrow B$$

If $A = \text{domain}$

then f is a total function ✱

otherwise f is a partial function ✱

Asymptotic Analysis

O: Upper Bounding Function

- **Def:** $f(n) = O(g(n))$ if $\exists c > 0$ and $n_0 > 0$ such that $0 \leq \underline{f(n)} \leq \underline{cg(n)}$ for all $\underline{n \geq n_0}$.

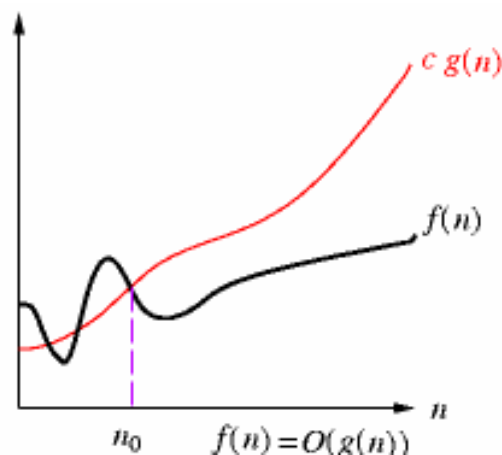
- Intuition: $f(n)$ " $\leq$ " $g(n)$ when we ignore constant multiples and small values of n .

- How to show O (Big-Oh) relationships?

– $f(n) = O(g(n))$ implies that $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c$ for some $\underline{c \geq 0}$, if the limit exists.

1. $3n^2 + n = O(n^2)$? $\checkmark$
2. $3n^2 + n = O(n)$? $\times n^2$
3. $3n^2 + n = O(n^3)$? $\checkmark$

n^2 tighter



$$f(n) = O(g(n)) \quad \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c \text{ for some } c \geq 0$$

$$\exists c > 0, n_0 > 0 \text{ s.t. } f(n) \leq cg(n) \text{ for all } n \geq n_0$$

$$\exists c > 0, n_0 > 0, \text{ s.t. } f(n) \leq cg(n) \text{ for all } n \geq n_0$$

Asymptotic Analysis

Ω : Lower Bounding Function

- **Def:** $f(n) = \Omega(g(n))$ if $\exists c > 0$ and $n_0 > 0$ such that $0 \leq cg(n) \leq f(n)$ for all $n \geq n_0$.

- Intuition: $f(n)$ “ $\geq$ ” $g(n)$ when we ignore constant multiples and small values of n .

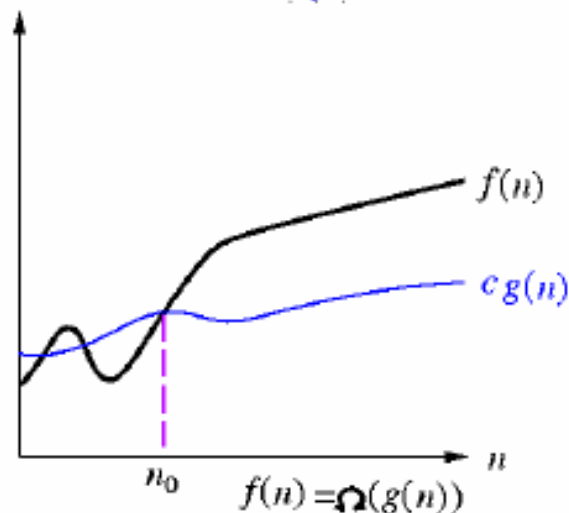
- How to show Ω (Big-Omega) relationships?

– $f(n) = \Omega(g(n))$ implies that $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = c$ for some $c \geq 0$ if the limit exists.

1. $3n^2 + n = \Omega(n^2)$?

2. $3n^2 + n = \Omega(n)$?

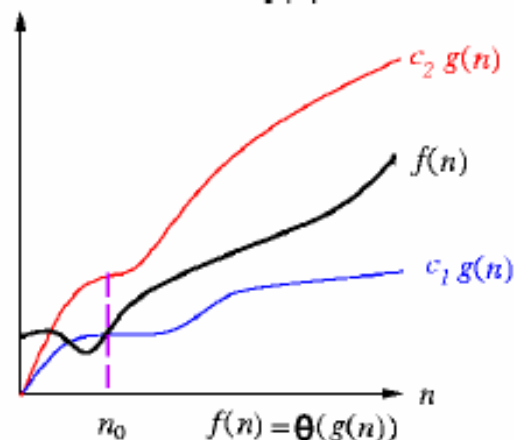
3. $3n^2 + n = \Omega(n^3)$?



Asymptotic Analysis

θ : Tightly Bounding Function

- **Def:** $f(n) = \theta(g(n))$ if $\exists c_1, c_2 > 0$ and $n_0 > 0$ such that $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$.
- Intuition: $f(n)$ “=” $g(n)$ when we ignore constant multiples and small values of n .
- How to show θ relationships?
 - Show both “big Oh” (O) and “Big Omega” (Ω) relationships.
 - $f(n) = \theta(g(n))$ implies that $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c$ for some $c > 0$, if the limit exists.



1. $3n^2 + n = \theta(n^2)$? ✓
2. $3n^2 + n = \theta(n)$? ✗
3. $3n^2 + n = \theta(n^3)$? ✗

Example 1.3

$$f(n) = 2n^2 + 3n,$$

$$g(n) = n^3,$$

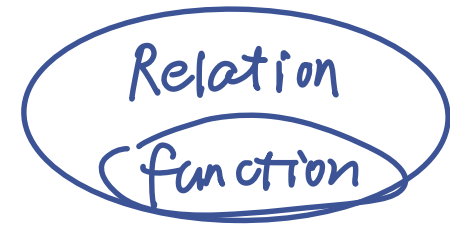
$$h(n) = 10n^2 + 100.$$

$$f(n) = O(g(n)),$$

$$g(n) = \Omega(h(n))$$

$$f(n) = \theta(h(n))$$

RELATIONS



Relations are more general than functions:

In a function, each element of the domains has **exactly one** associated ~~element in the range,~~

In a relation, there may be **several** such elements in the range.

$$R = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots\}$$

$$x_i R y_j$$

e. g. if $R = '>'$: $2 > 1, 3 > 2, 3 > 1$

Equivalence Relations ($\equiv$)

- Reflexive: $x R x$
- Symmetric: $x R y \longrightarrow y R x$
- Transitive: $x R y$ and $y R z \longrightarrow x R z$

Example: $R \equiv '='$

- $x = x$
- $x = y \longrightarrow y = x$
- $x = y$ and $y = z \longrightarrow x = z$

Example 1.4

On the set of nonnegative integers, we can define a relation

$$x \equiv y \text{ (同余3)}$$

If and only if

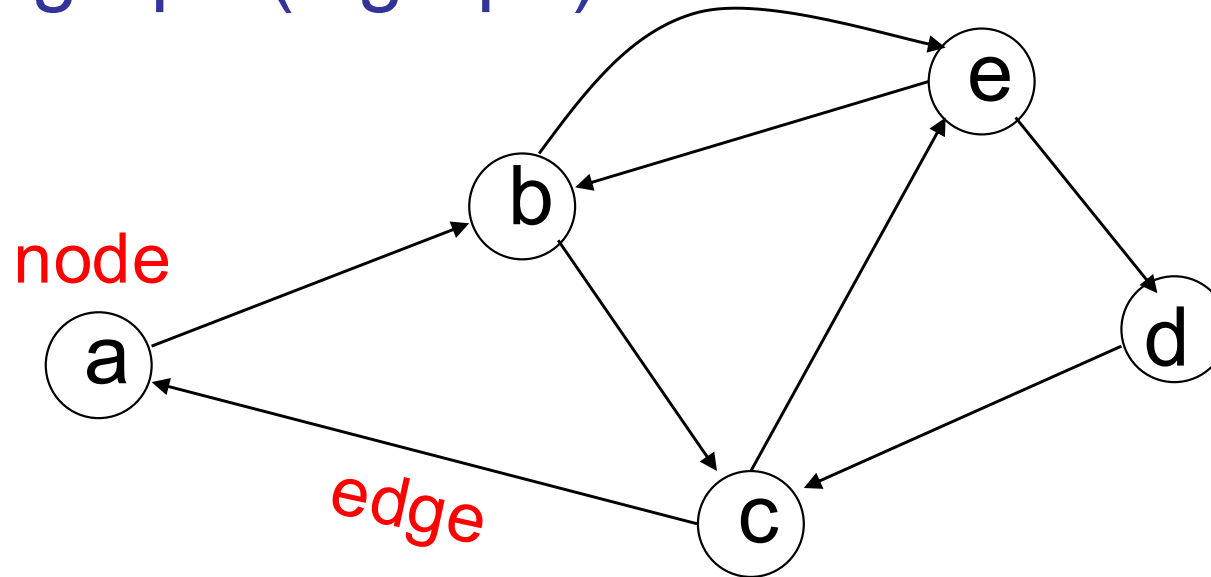
$$x \bmod 3 = y \bmod 3.$$

Then $2 \equiv 5$, $12 \equiv 0$, and $0 \equiv 36$.

Clearly this is an **equivalence relation**, as it satisfies reflexivity, symmetry, and transitivity.

GRAPHS

A directed graph (digraph)



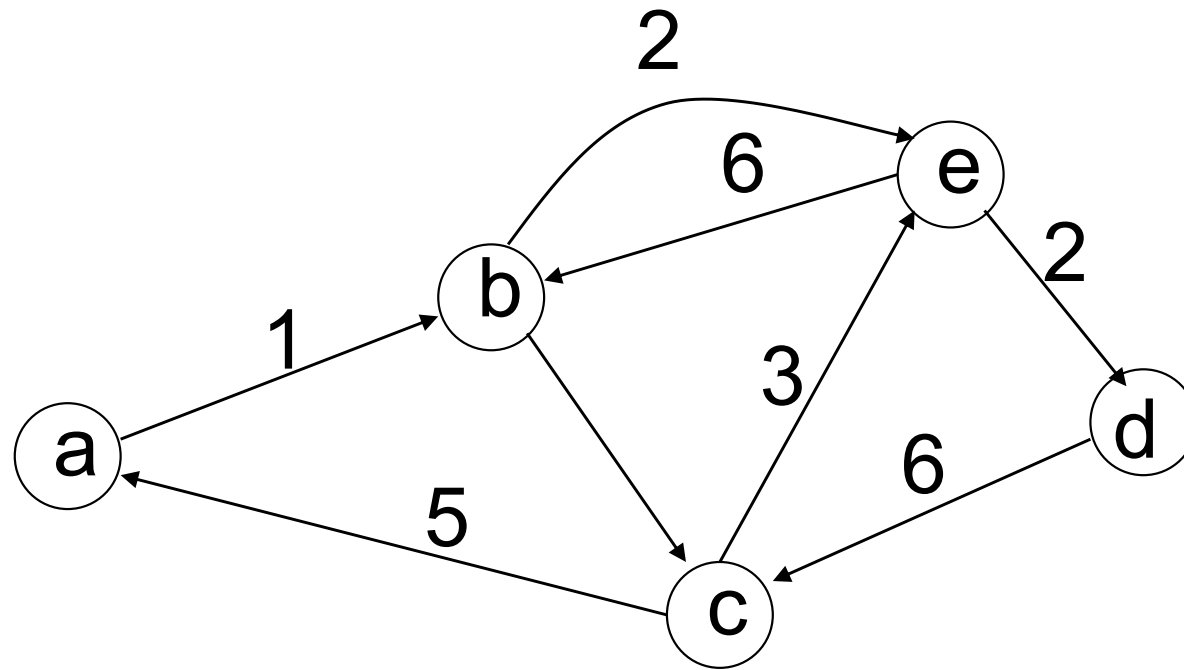
- Nodes (Vertices)

$$V = \{ a, b, c, d, e \}$$

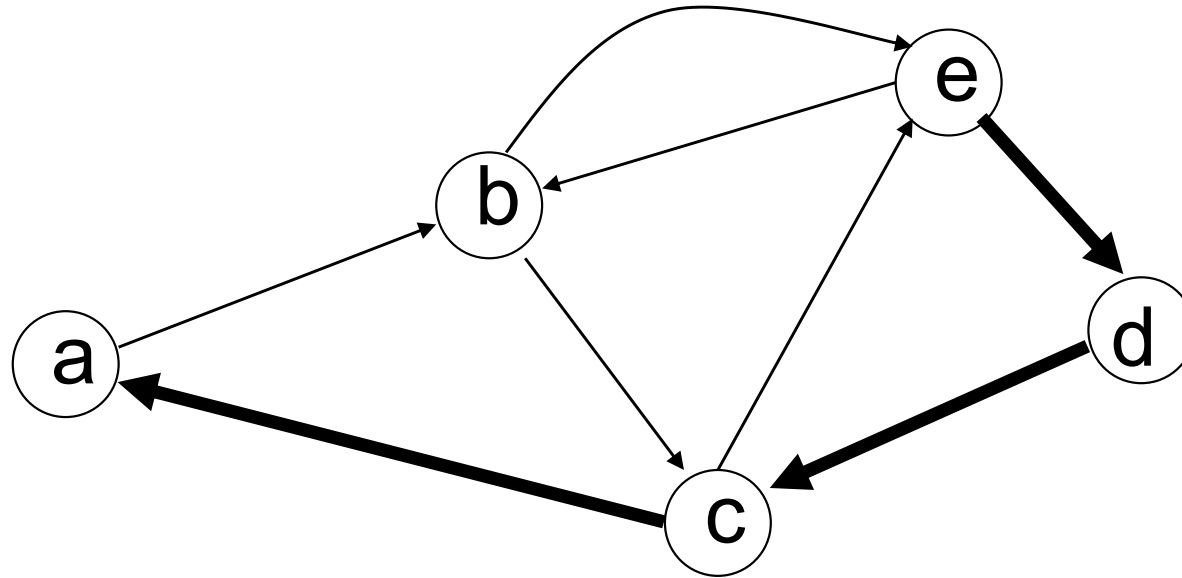
- Edges

$$E = \{ (a,b), (b,c), (b,e), (c,a), (c,e), (d,c), (e,b), (e,d) \}$$

Labeled Graph



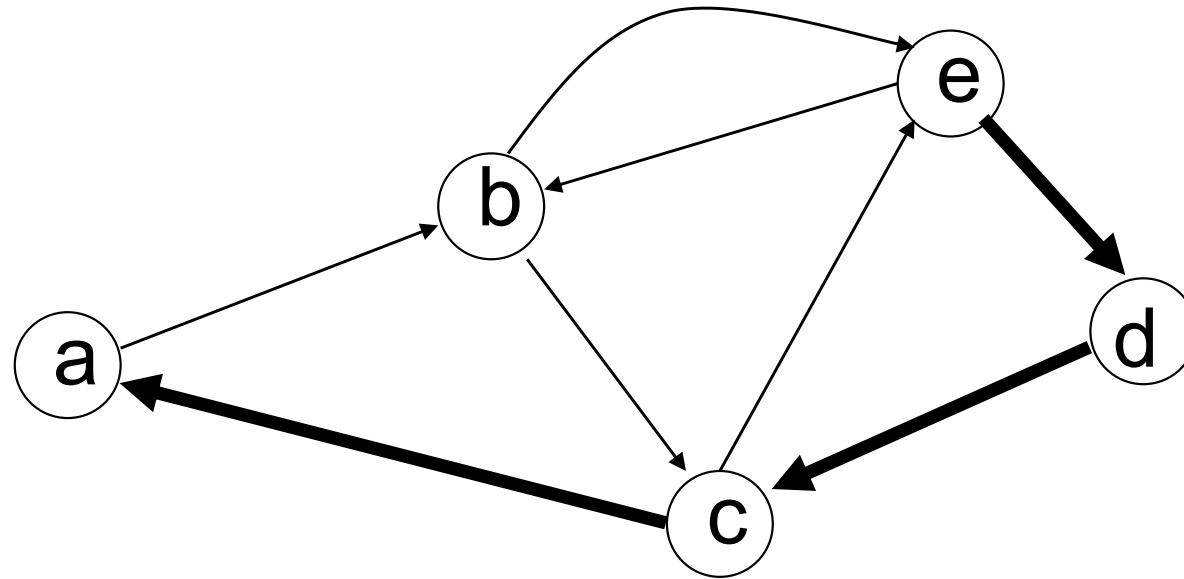
Walk



Walk is a sequence of adjacent edges

(e, d), (d, c), (c, a)

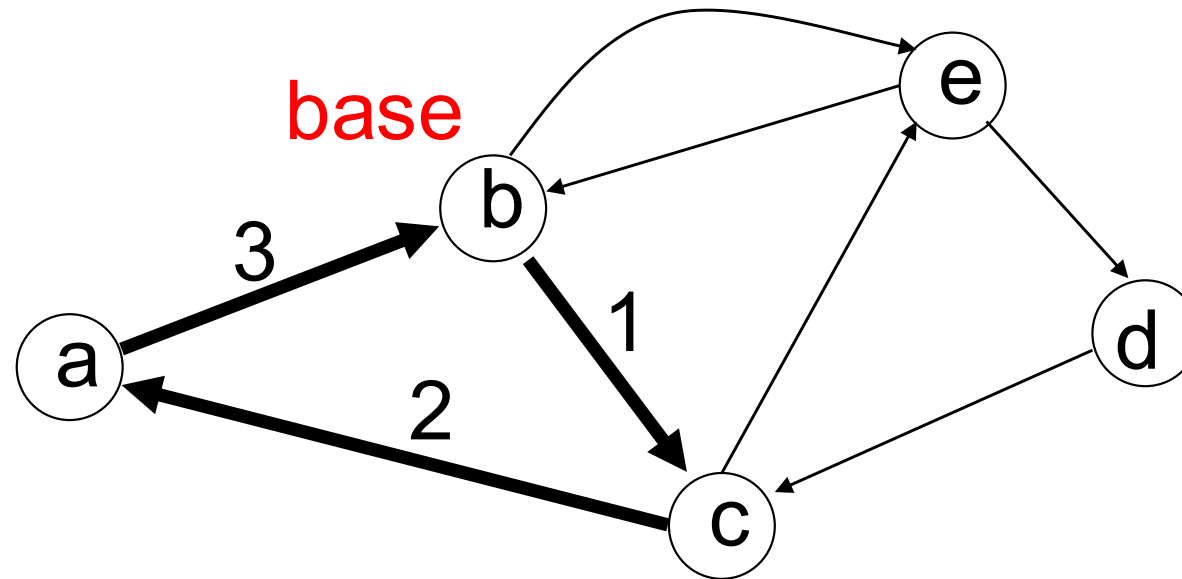
Path



No edge repeated
Path: a walk where no edge is repeated

Simple path: no node is repeated
No node repeated.

Cycle

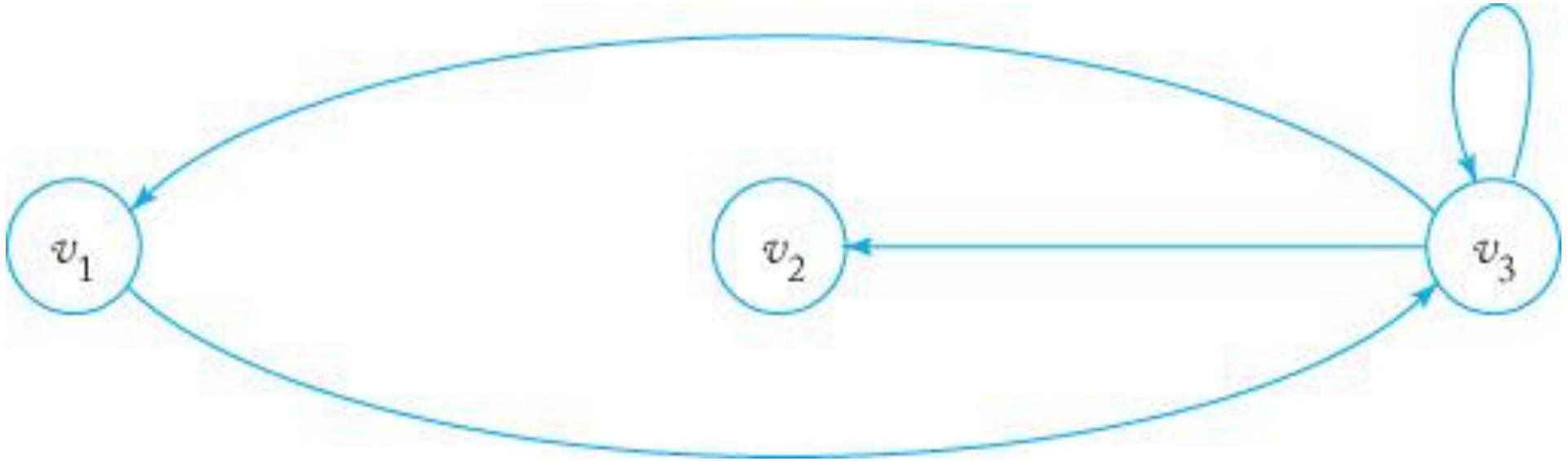


Cycle: a walk from a node (base) to itself

Simple cycle: only the base node is repeated
(start = End).

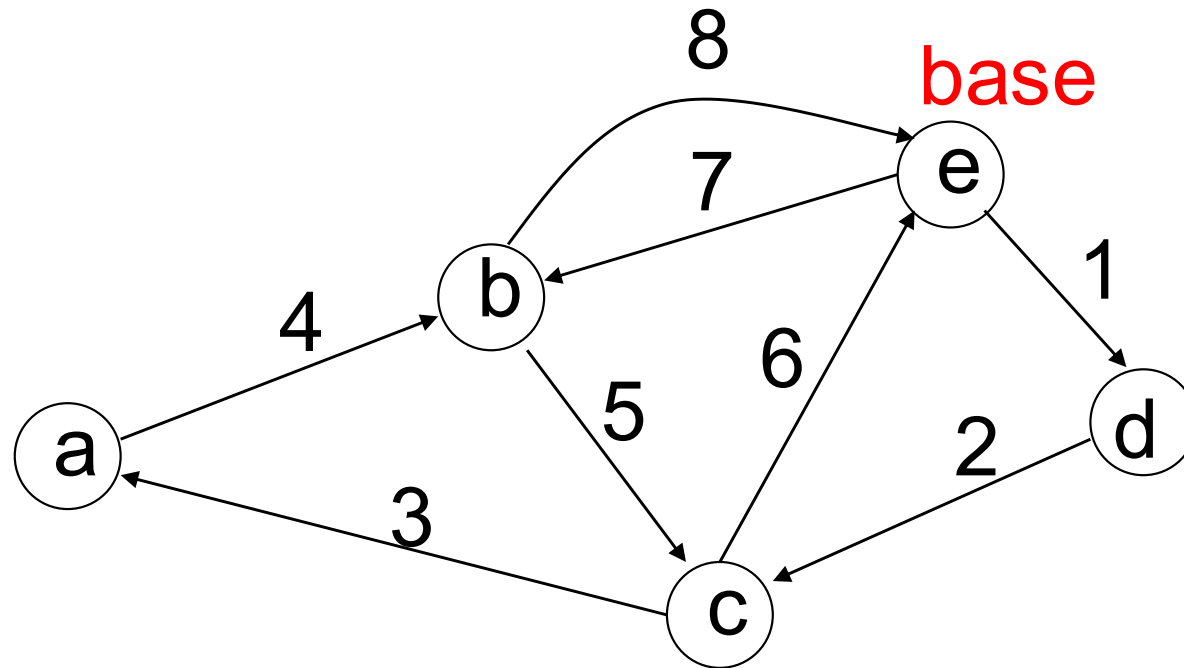
Loop

An edge from a vertex to itself



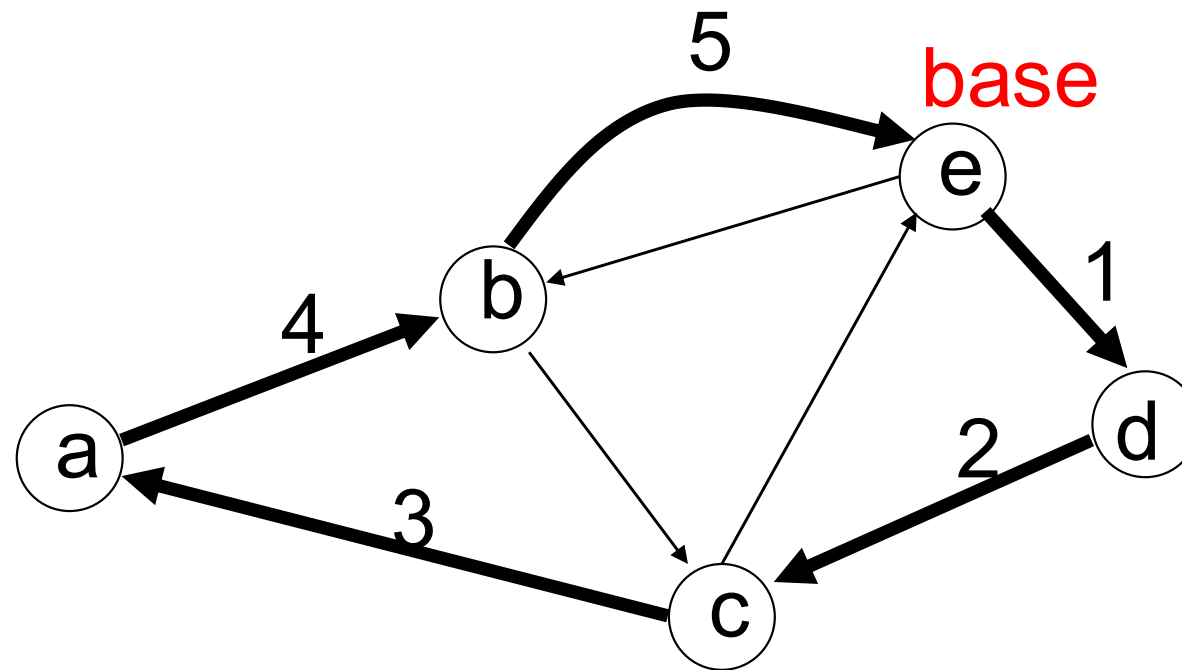
- $(v_1, v_3), (v_3, v_2)$ is a simple path from v_1 to v_2
- $(v_1, v_3), (v_3, v_3), (v_3, v_1)$ is a cycle (not simple one)
- There is a loop on vertex v_3

Euler Tour



A cycle that contains each edge once

Hamiltonian Cycle

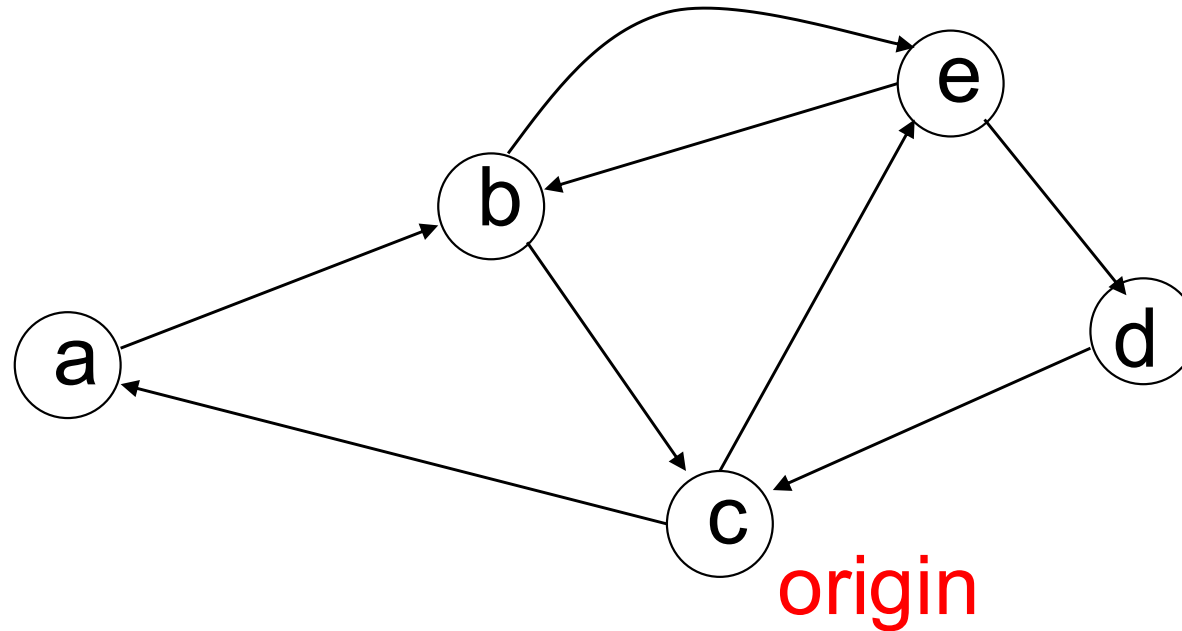


A simple cycle that contains all nodes

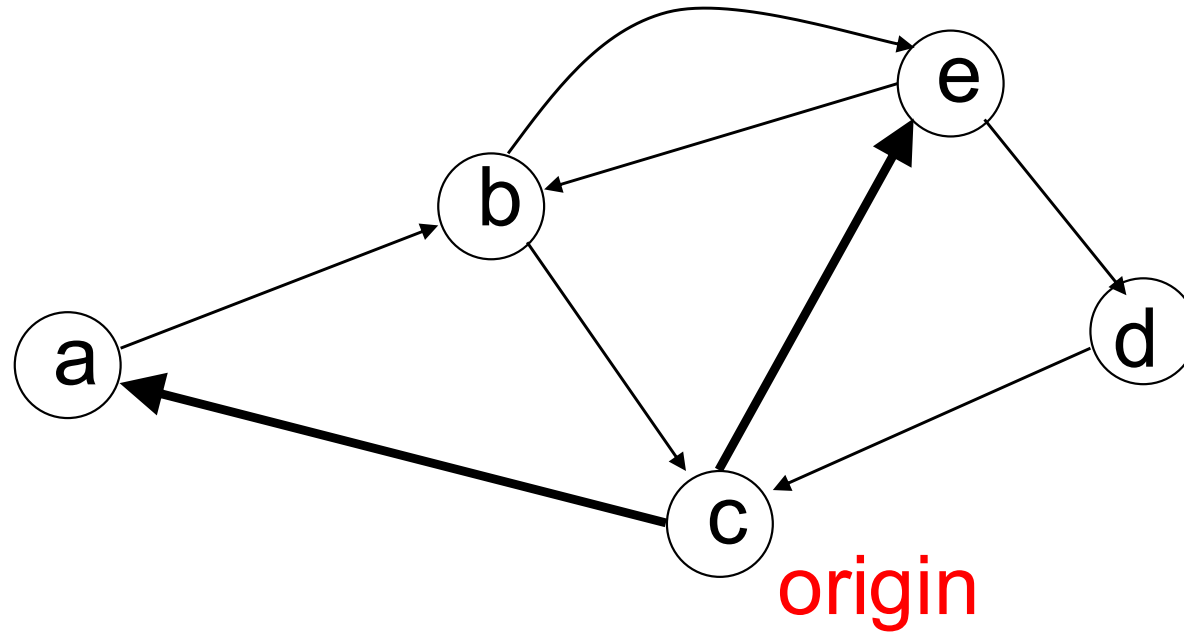
Exercise

Finding All Simple Paths

No nodes repeated.



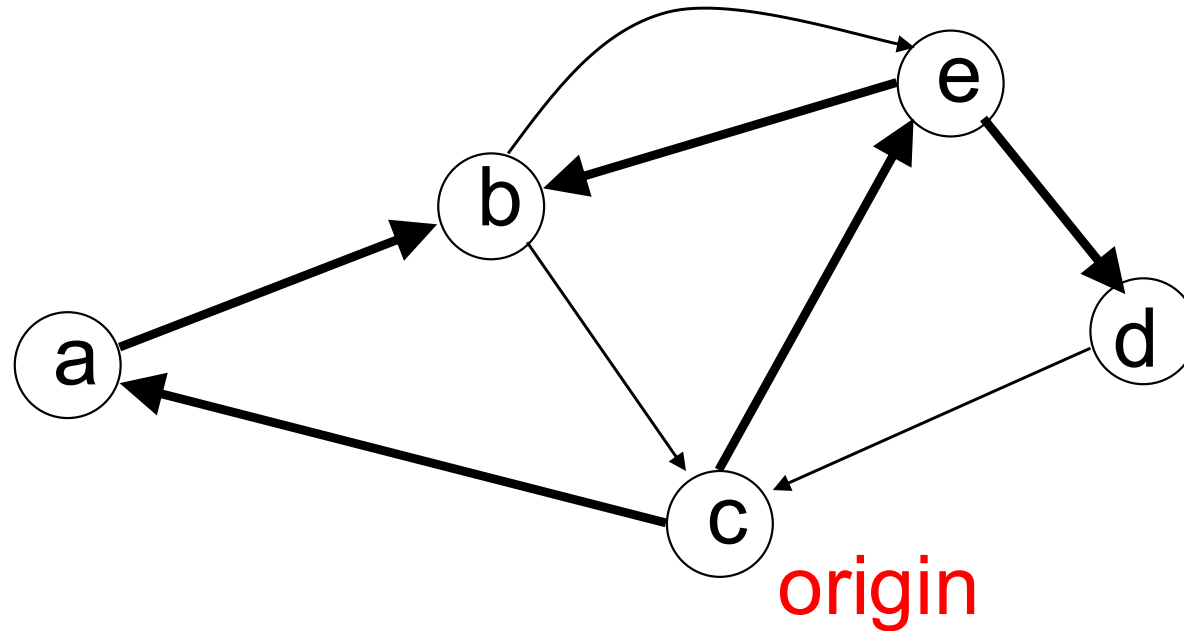
Step 1



(c, a)

(c, e)

Step 2



(c, a)

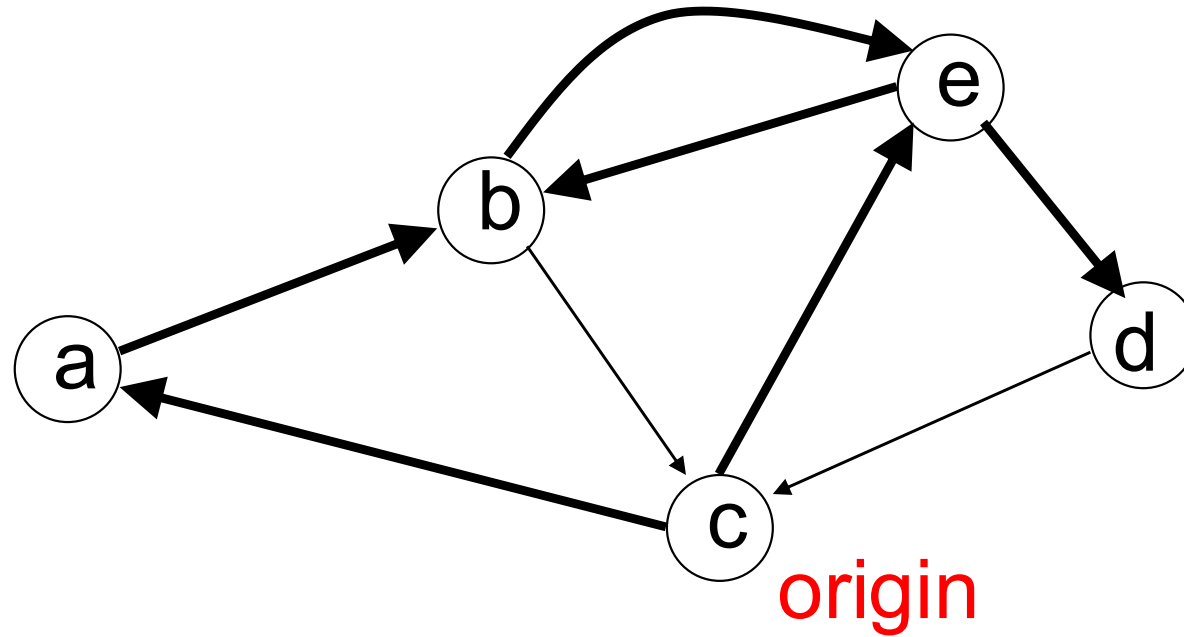
(c, a), (a, b)

(c, e)

(c, e), (e, b)

(c, e), (e, d)

Step 3



(c, a)

(c, a), (a, b)

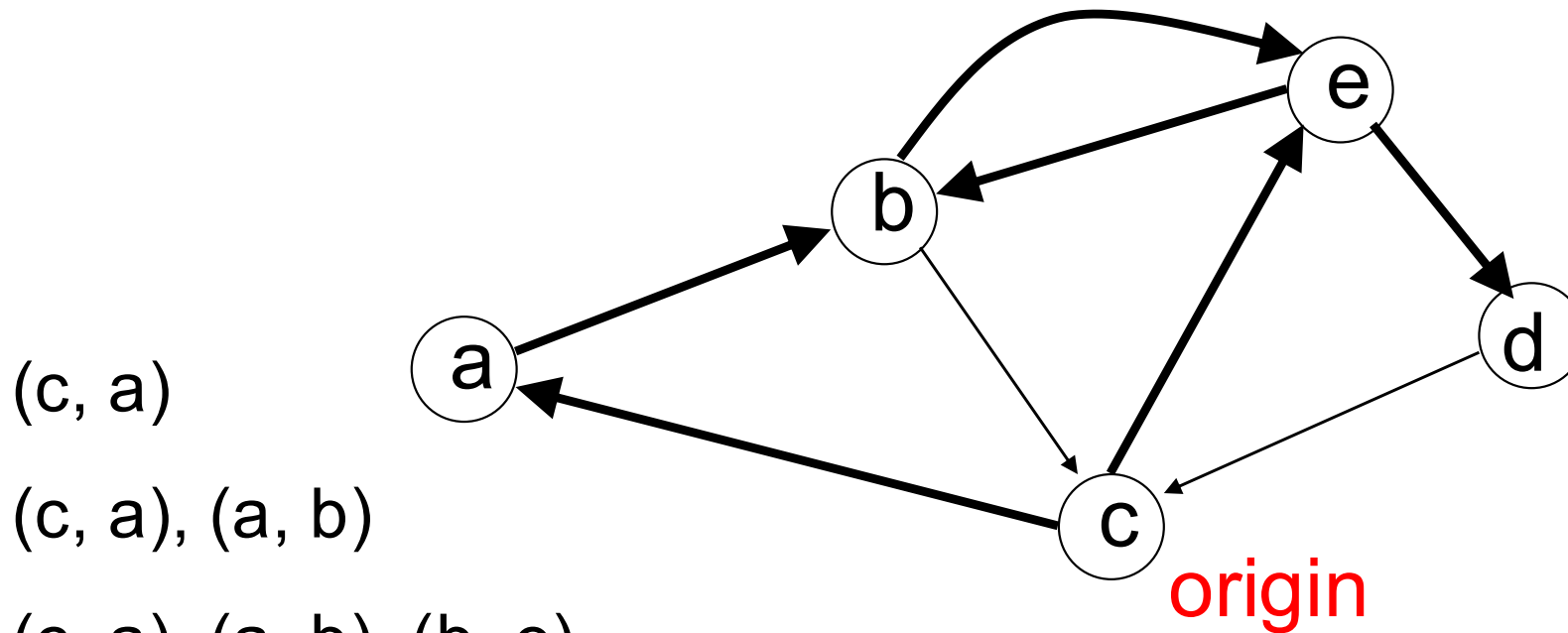
(c, a), (a, b), (b, e)

(c, e)

(c, e), (e, b)

(c, e), (e, d)

Step 4



(c, a)

(c, a), (a, b)

(c, a), (a, b), (b, e)

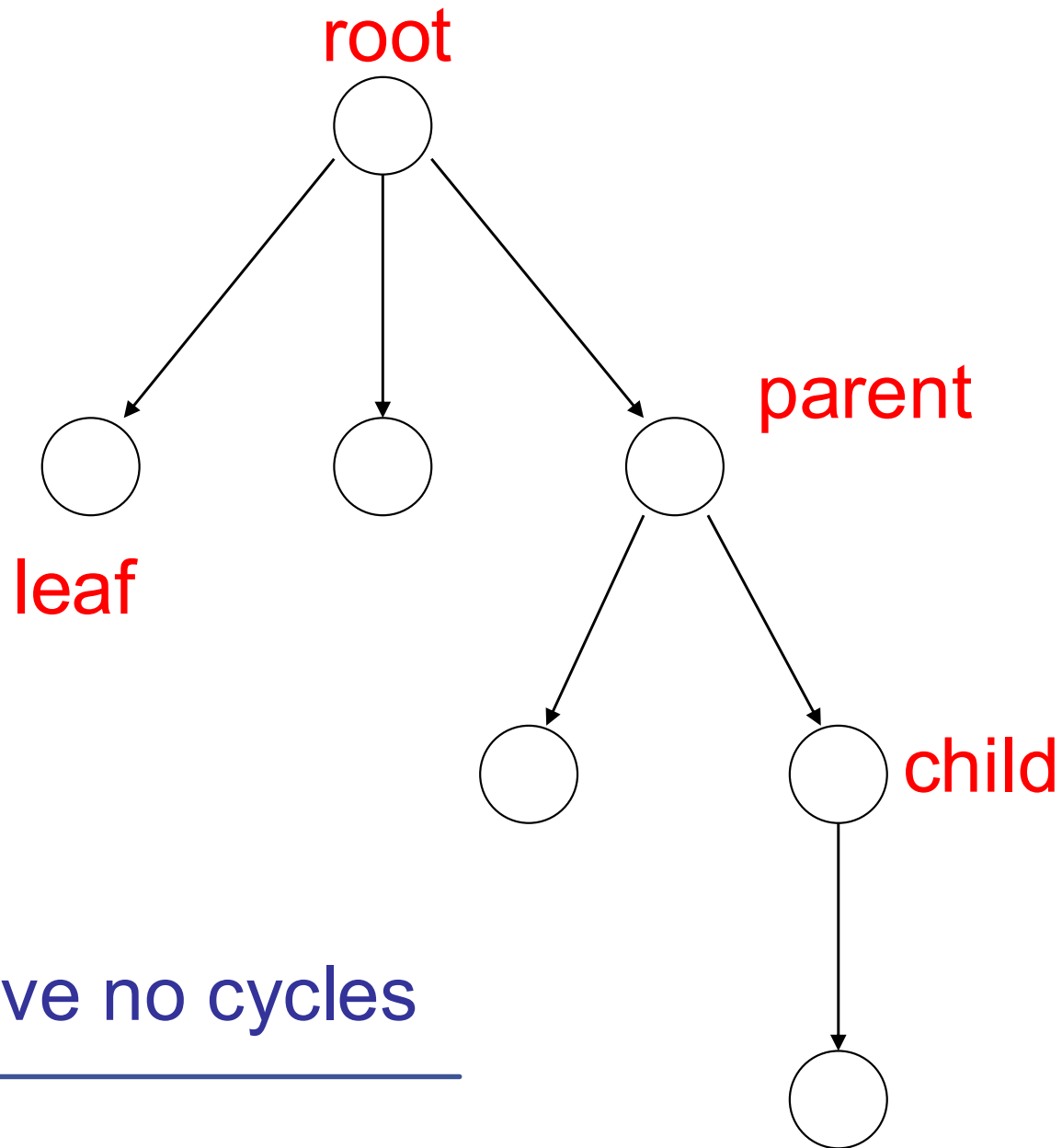
(c, a), (a, b), (b, e), (e, d)

(c, e)

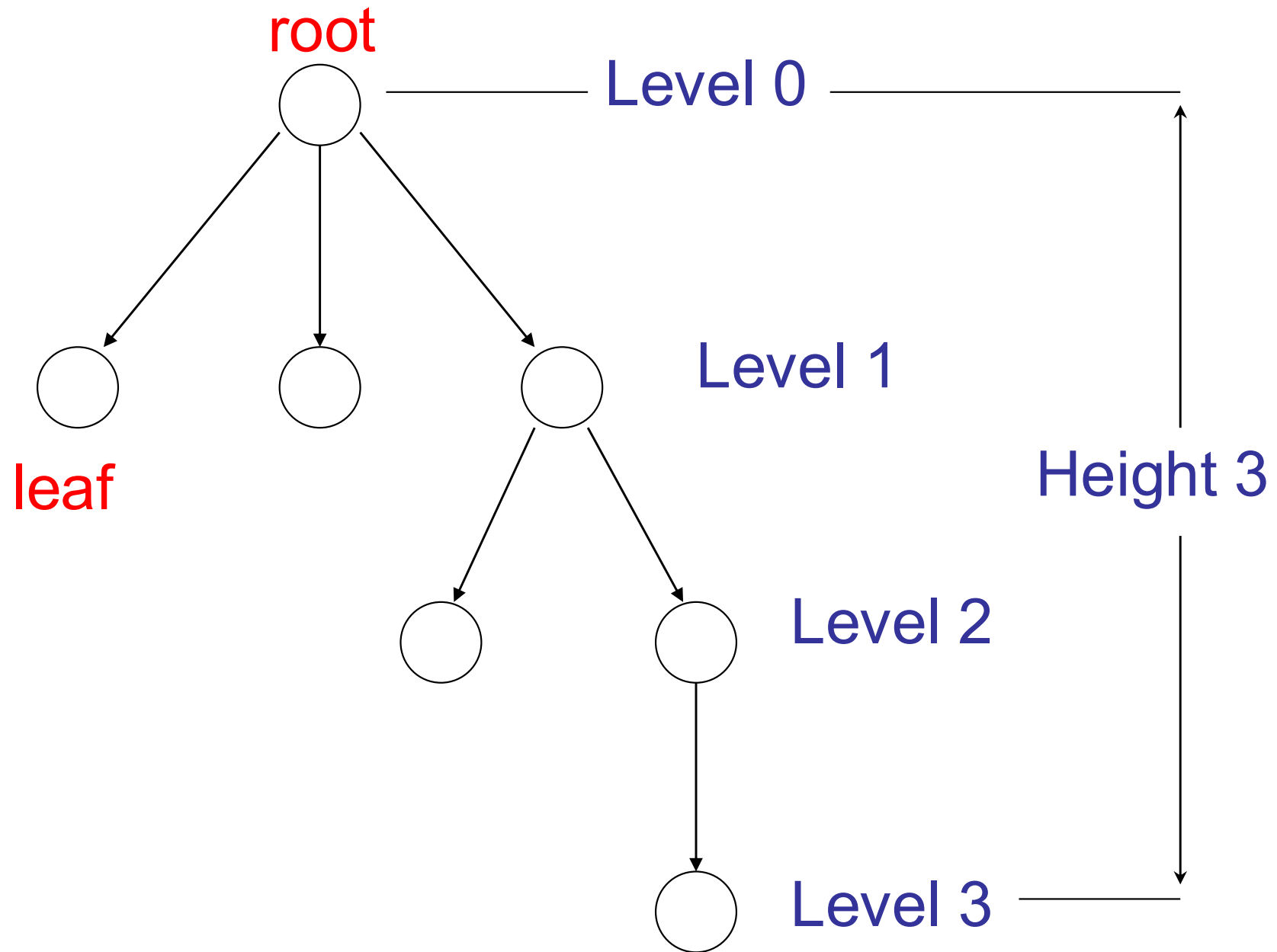
(c, e), (e, b)

(c, e), (e, d)

Trees



Trees have no cycles



Proof Techniques

- Direct/Constructive Proof
- Proof by Induction
- Proof by Contradiction

Direct/Constructive Proof

- If X , then Y
- Assume X is true, show directly that Y is true.
(e.g. X = it rains, Y = sidewalk will wet)
 - Example:
 - For integers a, b : If a and b are odd, then ab is odd.

Given: a and b are odd integers. There exists integer x such that $a = 2x + 1$
 y $b = 2y + 1$

Mult prove $ab = 2z + 1$

$$ab = (2x + 1)(2y + 1) = 4xy + 2(xy) + 1 \Rightarrow \text{odd.}$$

Direct/Constructive Proof

- Perform the multiplication directly

- $$\begin{aligned} ab &= (2x + 1)(2y + 1) \\ &= 4xy + 2x + 2y + 1 \\ &= 2(2xy + x + y) + 1 \end{aligned}$$

So $z = 2xy + x + y$

Not only did you prove that a z exists, you constructed an “algorithm” for generating this z .

This is an example of a constructive proof.

Induction

We have statements $P_1, P_2, P_3, \dots$

If we know

- for some b that $P_1, P_2, \dots, P_b$ are true
- for any $k \geq b$ that

$$P_1, P_2, \dots, P_k \text{ imply } P_{k+1}$$

Then

Every P_i is true

Proof by Induction

- Inductive basis

Find $P_1, P_2, \dots, P_b$ which are true

- Inductive hypothesis

Let's assume $P_1, P_2, \dots, P_k$ are true,
for any $k \geq b$

- Inductive step

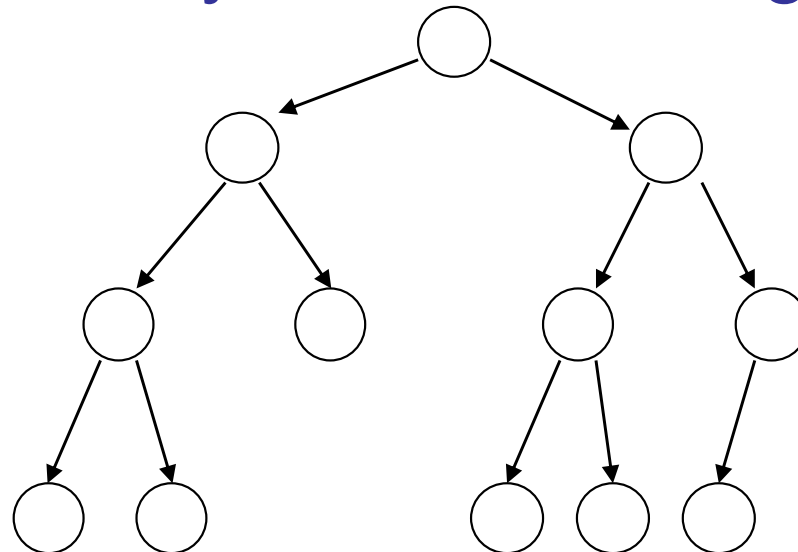
Show that P_{k+1} is true

Example 1.5

Theorem: A binary tree of height n has at most 2^n leaves.

Proof by induction:

let $L(i)$ be the maximum number of leaves of any subtree at height i



We want to show: $L(i) \leq 2^i$

- Inductive basis

$L(0) = 1$ (the root node) 

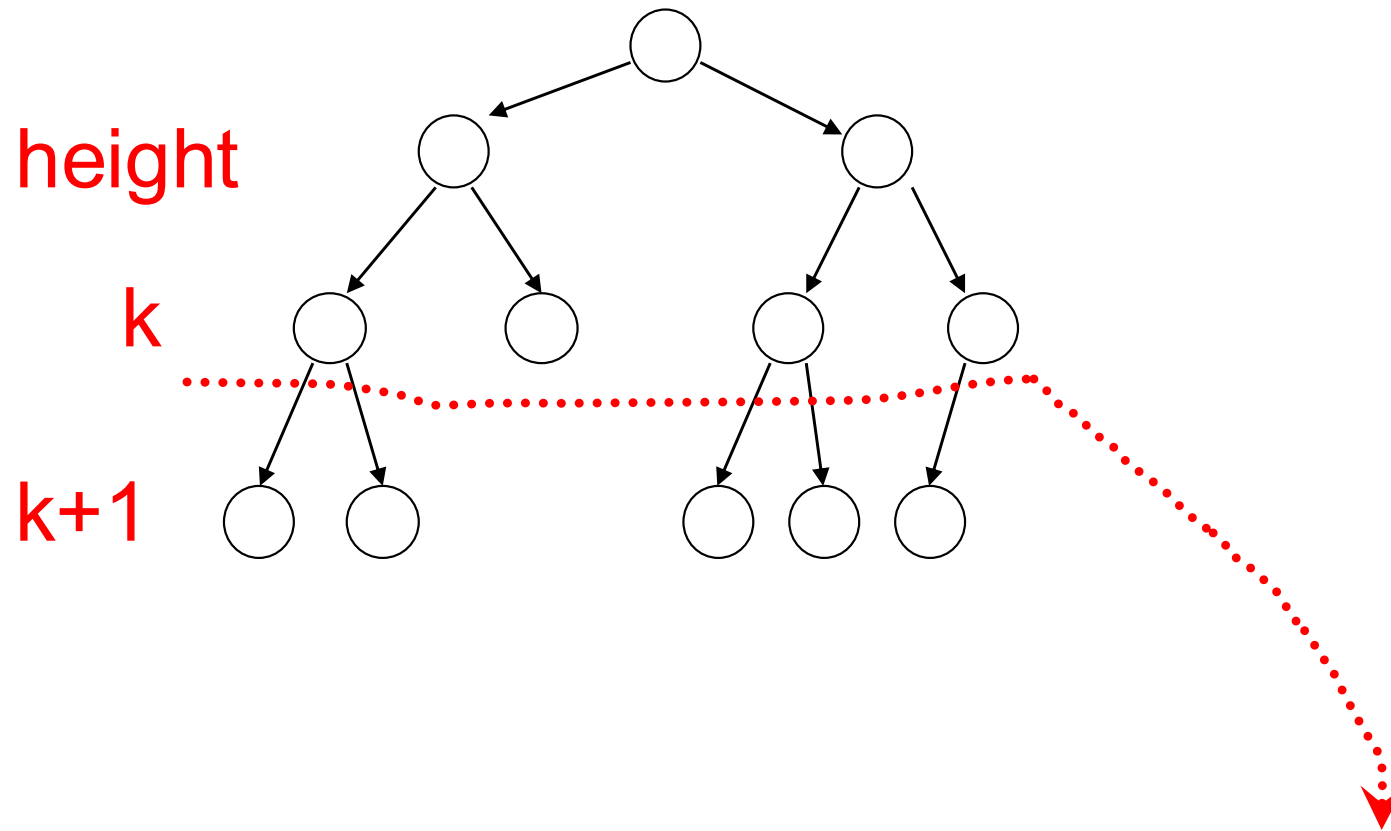
- Inductive hypothesis

Let's assume $L(i) \leq 2^i$ for all $i = 0, 1, \dots, k$

- Induction step

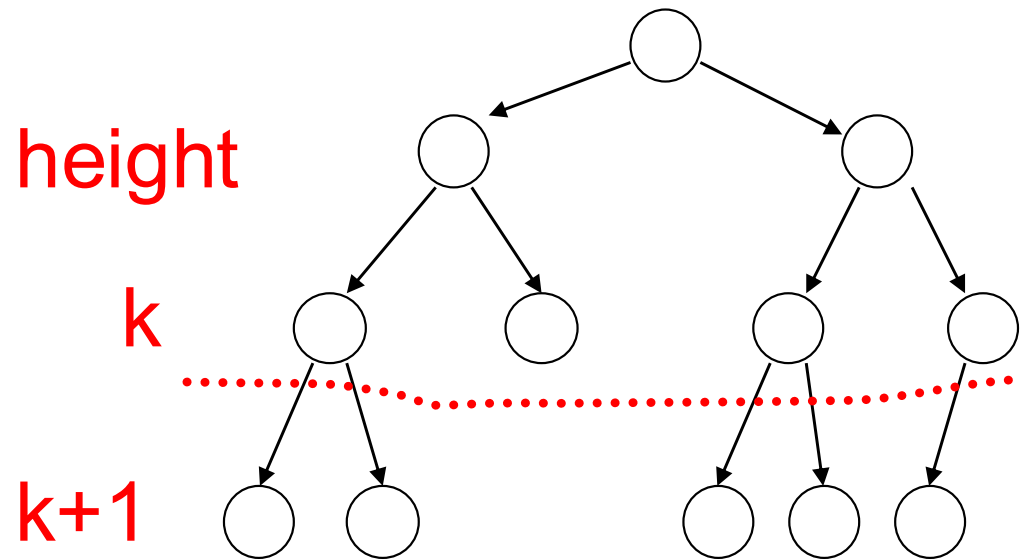
we need to show that $L(k + 1) \leq 2^{k+1}$

Induction Step



From Inductive hypothesis: $L(k) \leq 2^k$

Induction Step



$$L(k) \leq 2^k$$

→ Binary tree: Every node has at most two children nodes.

$$L(k+1) \leq 2 * L(k) \leq 2 * 2^k = 2^{k+1}$$

(we add at most two nodes for every leaf of level k)

Remark

Recursion is another thing

Example of recursive function:

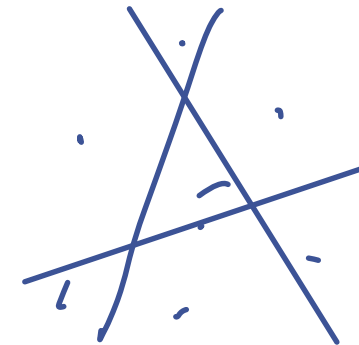
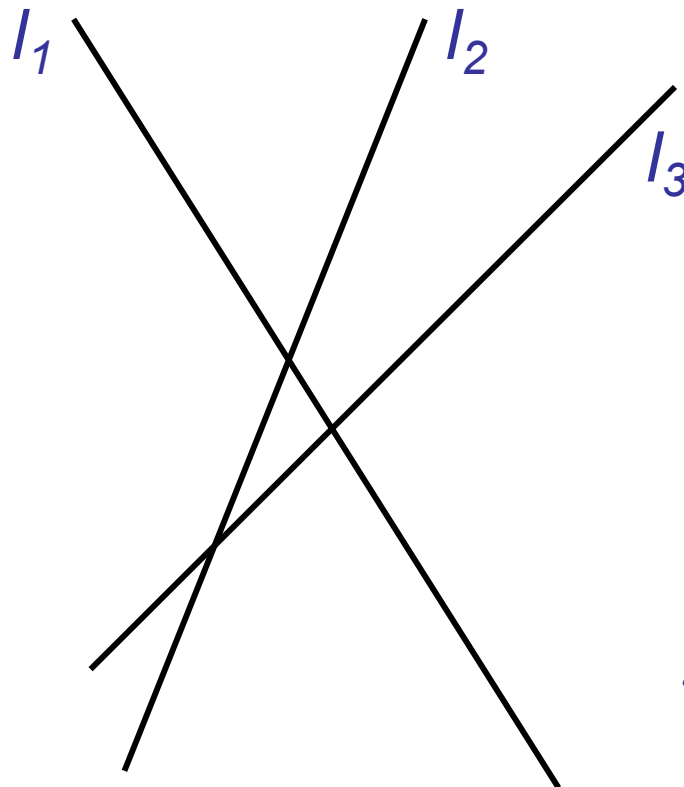
$$f(n) = f(n-1) + f(n-2)$$

$$f(0) = 1, \quad f(1) = 1$$

Example 1.6

A set $l_1, l_2, \dots, l_n$ of mutually intersecting straight lines divides the plane into a number of separated regions

1 line $\rightarrow$ 2 regions, 2 lines $\rightarrow$ 4 regions, 3 lines $\rightarrow$ 7 regions

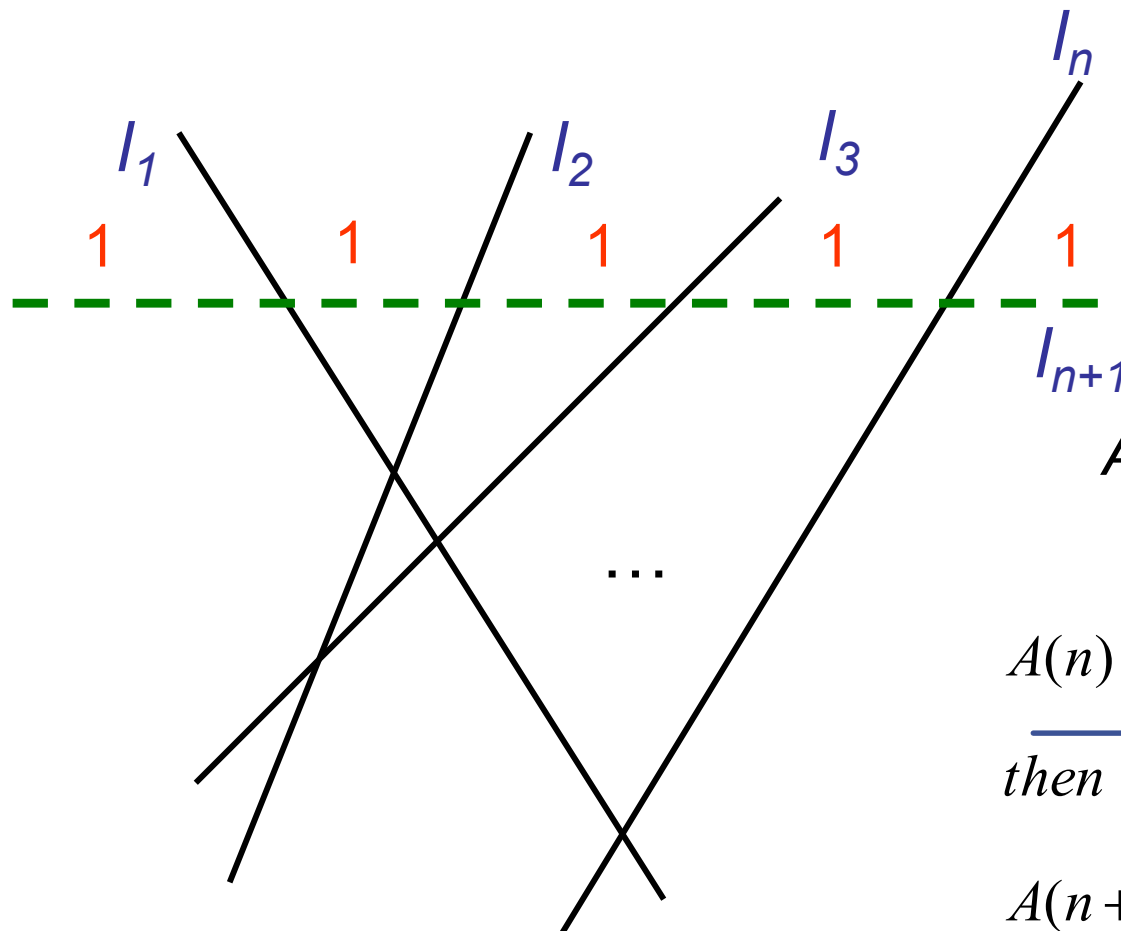


Solve it recursively!!

增加 one line:

$N(\text{Add nodes}) \Leftrightarrow N(\text{Add regions})$

Example 1.6



Let $A(n)$ denote the number of regions generated by n lines

$$A(n+1) = A(n) + n + 1, n = 1, 2, \dots,$$

$$A(1) = 2, A(2) = 4, A(3) = 7, A(4) = 11$$

$$A(n) = \frac{n(n+1)}{2} + 1$$

↓ 归纳法

then

$$A(n+1) = \frac{n(n+1)}{2} + 1 + n + 1 = \frac{(n+1)(n+2)}{2} + 1$$

Proof by Contradiction

↳ for complex prob.

We want to prove that a statement P is true

- we assume that P is false
- then we arrive at an incorrect conclusion
- therefore, statement P must be true

Example 1.7

Theorem: $\sqrt{2}$ is not rational

Proof:

Assume by contradiction that it is rational

$$\sqrt{2} = n/m$$

n and m have no common factors

We will show that this is impossible

$$\sqrt{2} = n/m \quad \longrightarrow \quad 2 m^2 = n^2$$

$$2 = n^2/m^2 \Rightarrow 2m^2 = n^2$$

Therefore, n^2 is even $\longrightarrow$ n is even
 $n = 2k$

$$2 m^2 = 4k^2 \quad \longrightarrow \quad m^2 = 2k^2 \quad \longrightarrow \quad \begin{array}{l} \text{m is even} \\ \underline{m = 2p} \end{array}$$

Thus, m and n have common factor 2

Contradiction! ✓

Outline



Mathematical Preliminaries and Notation



Three Basic Concepts



Some Applications

Three Basic Concepts

- Languages
- Grammars
- Automata (will discuss in Chap. 2)

A language is a set of **strings**

String: A sequence of symbols from the alphabet

- Examples: “**cat**”, “**dog**”, “**house**”, ...

- Defined over an **alphabet**:

$$\overset{\text{Sigma.}}{\Sigma} = \{a, b, c, \dots, z\}$$

Alphabets and Strings

阿巴诺.

$$\Sigma = \{a, b\}$$

a

ab

abba

baba

aaabbbbaabab

$$u = ab$$

$$v = bbbbaaa$$

$$w = abba$$

String Operations

$$w = a_1 a_2 \cdots a_n$$

abba

$$v = b_1 b_2 \cdots b_m$$

bbbbaaa

⊇ Concatenation

$$wv = a_1 a_2 \cdots a_n b_1 b_2 \cdots b_m$$

abbabbbaaa

$$w = a_1 a_2 \cdots a_n$$

ababaaaabbb

Reverse

$$w^R = a_n \cdots a_2 a_1$$

bbbaaababab

String Length

$$w = a_1 a_2 \cdots a_n$$

- Length: $|w| = n$
- Examples: $|abba| = 4$
 $|aa| = 2$
 $|a| = 1$

Length of Concatenation

$$|uv| = |u| + |v|$$

- Example: $u = aab$, $|u| = 3$
 $v = abaab$, $|v| = 5$

$$|uv| = |aababaab| = 8$$

$$|uv| = |u| + |v| = 3 + 5 = 8$$

Empty String

- A string with no letters: λ *lambda*

- Observations: $|\lambda| = 0$

$$\lambda w = w\lambda = w$$

Concatenation

$$\lambda abba = abba\lambda = abba$$

Substring

- Substring of string:
 - a subsequence of consecutive characters

連続的

String

Substring

abbab

ab

abbab

abba

abbab

b

abbab

bbab

Prefix and Suffix

abbab

Prefixes

Suffixes

λ

a

ab

abb

abba

abbab

abbab

bbab

bab

ab

b

λ

$w = uv$

prefix
前綴

suffix
后綴

Another Operation

$$w^n = \underbrace{ww \cdots w}_n$$

- Example: $(abba)^2 = abbaabba$
Self-concatination.

- Definition: $w^0 = \lambda$

$$(abba)^0 = \lambda$$

The * Operation

Σ^* : the set of all possible strings from
alphabet Σ

$\Sigma = \{a, b\}$ Σ^* : 求所有可能组合.

$\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$

The + Operation

Σ^+ : the set of all possible strings from alphabet Σ except λ

$$\Sigma = \{a, b\}$$

→ All possible combinations

$$\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$$

→ 非空 combinations

$$\Sigma^+ = \Sigma^* - \lambda$$

$$\Sigma^+ = \{a, b, aa, ab, ba, bb, aaa, aab, \dots\}$$

Languages

A language is any subset of Σ^*

Example: $\Sigma = \{a, b\}$

$$\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, \dots\}$$

Languages: $\{\lambda\}$
(Finite)

$$\{a, aa, aab\}$$

$$\{\lambda, abba, baba, aa, ab, aaaaaa\}$$

Note that:

Sets

$$\emptyset = \{\} \neq \{\lambda\}$$

Set size

$$|\{\}| = |\emptyset| = 0$$

Set size

$$|\{\lambda\}| = 1$$

String length

$$|\lambda| = 0$$

Another Example

- An **infinite** language $L = \{a^n b^n : n \geq 0\}$

λ

ab

$aabb$

$aaaaabbbbb$

$\in L$

a, b: must same numbers.

$\uparrow$
 $abb \notin L$

Operations on Languages

- The usual set operations

$$\{a, ab, aaaa\} \cup \{bb, ab\} = \{a, ab, bb, aaaa\}$$

$$\{a, ab, aaaa\} \cap \{bb, ab\} = \{ab\}$$

$$\{a, ab, aaaa\} - \{bb, ab\} = \{a, aaaa\}$$

- Complement: $\bar{L} = \Sigma^* - L$ 字母表 combinations
- 原sets.

$$\overline{\{a, ba\}} = \{\lambda, b, aa, ab, bb, aaaa, \dots\}$$

Reverse

Definition: $L^R = \{w^R : w \in L\}$
↘ 对 set 整体 reverse = 对中间所有元素 reverse.

Examples: $\{ab, aab, baba\}^R = \{ba, baa, abab\}$

$$L = \{a^n b^n : n \geq 0\}$$



$$L^R = \{b^n a^n : n \geq 0\}$$

Concatenation

Definition: $L_1 L_2 = \{xy : x \in L_1, y \in L_2\}$

↪ prefix + suffix

Example: $\{a, ab, ba\}\{b, aa\}$

$= \{ab, aaa, abb, abaa, bab, baaa\}$

Another Operation

- Definition: $L^n = \underbrace{LL \cdots L}_n$

$$\{a,b\}^3 = \{a,b\}\{a,b\}\{a,b\} = \{aa, ab, ba, bb\}\{a,b\}$$
$$= \{aaa, aab, aba, abb, baa, bab, bba, bbb\}.$$

- Special case: $L^0 = \{\lambda\}$

$$\{a, bba, aaa\}^0 = \{\lambda\}$$

More Examples

$$L = \{a^n b^n : n \geq 0\}$$

$$L^2 = \{a^n b^n a^m b^m : n, m \geq 0\}$$

m, n 不一定相等

$$aabbbaaabb \in L^2$$

Note that n and m in the above are unrelated

Star-Closure (Kleene *)

克林恩星闭包
Star operation

- Definition: $L^* = L^0 \cup L^1 \cup L^2 \dots$

- Example:
$$\{a, bb\}^* = \left\{ \begin{array}{l} \lambda, \\ a, bb, \\ aa, abb, bba, bbbb, \\ aaa, aabb, abba, abbbb, \dots \end{array} \right\}$$

Positive Closure

- Definition: $L^+ = L^1 \cup L^2 \cup \dots$
 $= \underbrace{L^* - \{\lambda\}}$

Note: L 为 set of strings. i.e. A kind of language

$$\{a, bb\}^+ = \left\{ \begin{array}{l} a, bb, \\ aa, abb, bba, bbbb, \\ aaa, aabb, abba, abbbb, \dots \end{array} \right\}$$

Grammars

↳ How symbols are manipulated.

Wikipedia says:

- Languages can be described as a system of symbols and the grammars (rules) by which the symbols are manipulated
- Grammar is the study of **rules** governing the use of language.

Grammars

- Think back to your days of learning English
- Rules for constructing a simple sentence

Sentence = noun phrase + verb phrase

- Noun phrase =
 - Name (Joe)
 - Article + noun (the car)
- Verb Phrase =
 - Verb (runs)
 - Verb + prepositional phrase
- Prepositional Phrase =
 - Preposition + noun phrase (from the car)

Grammars

- Look at the sentence. Is this grammatically correct?

Joe runs from the car.

Sentence = noun phrase + verb phrase
= noun + verb phrase
= Name + verb phrase
= Joe + verb phrase
= Joe + verb + prepositional phrase
= Joe + verb + preposition + noun phrase
= Joe + verb + from + noun phrase
= Joe + verb + from + article + noun
= Joe + verb + from + article + noun
= Joe + verb + from + the + car
= Joe + runs + from + the + car

Valid sentence!

Definition 1.1

- A grammar G is defined as a 4-tuple:

$$G = (V, T, S, P)$$

where

↓ 可以操作 production rule 的对象.

Ex:

$$G = (\{S\}, \{a, b\}, S, P)$$

- V is a finite set of variables

means Alphabet. ←

- T is a finite set of terminals

- $S \in V$, called start variable

- P is a finite set of production rules

$$P: S \rightarrow aSb, \\ S \rightarrow \lambda$$

(production rules)

read: S can be replaced by aSb

or: S can be translated as aSb

Grammars

$$G = (V, T, S, P)$$

- Let's formalize this a bit:

Production rules $(x \rightarrow y)$ where $x \in (V, T)^+$ *cannot be λ .*
 $y \in (V, T)^*$

They specify how the grammar transforms one string into another

- We say that γ can be derived from α in one step:

$A \rightarrow \beta$ is a production rule

$$\alpha = \alpha_1 A \alpha_2$$

$$\gamma = \alpha_1 \beta \alpha_2$$

$$\alpha \Rightarrow \gamma$$

- We write $\alpha \xRightarrow{*} \gamma$ if γ can be derived *起源于* from α (or say α derives γ) in zero or more steps.

Definition 1.2

- Let $G = (V, T, S, P)$ be a grammar. Then the set

$$L(G) = \{w \in T^* : S \Rightarrow^* w\}$$
is the **language** generated by G

- If $w \in L(G)$, then the sequence

$$S \Rightarrow w_1 \Rightarrow w_2 \Rightarrow \dots \Rightarrow w_n \Rightarrow w \text{ (sentence)}$$

is a derivation of the sentence w . sentential forms.

- $S, w_1, w_2, \dots, w_n$ are called **sentential forms** (half-done sentence)

Example 1.11

$$G = (V, T, S, P)$$

Consider the grammar:

$$G = (\{S\}, \{a, b\}, S, P)$$

$L(G)?$

With P given by

$$L(G) = \{a^n b^n : n \geq 0\}$$

$$S \rightarrow aSb, \textcircled{1}$$

$$S \rightarrow \lambda \textcircled{2}$$

Then

$$S \Rightarrow aSb \Rightarrow \underbrace{aaSbb}_{\text{replace (Follow } \textcircled{1})}} \Rightarrow aabb,$$

replace

So we can write

$$\underline{S \overset{*}{\Rightarrow} aabb} \quad (\text{Follow } \textcircled{2})$$

- String $aabb$ is a sentence in the language generated by G
- $aaSbb$ is a sentential form

Example 1.12

$$G = (V, T, S, P)$$

Find a grammar that generates

$$L = \{a^n b^{n+1} : n \geq 0\}$$

abb, aabbb.
 $\{S\}$, $T = \{a, b\}$

Previous example

$G = (\{S\}, \{a, b\}, S, P)$ with $P: S \rightarrow aSb, S \rightarrow \lambda$

All we need to do is generate an extra b * Introduce an extra variable A ,

$G = (\{S, A\}, \{a, b\}, S, P)$, with productions

$S \rightarrow Ab$, Introduce an extra b.

$A \rightarrow aAb$,
 $A \rightarrow \lambda$ } concatenate a b. + therefore generates $a^n b^n$

Or: $S \rightarrow aSb, S \rightarrow b$.

Example 1.13

$$G = (V, T, S, P)$$

Consider the grammar:

$$G = (\{S\}, \{a, b\}, S, P)$$

With P given by

$$S \rightarrow SS,$$

$$S \rightarrow \lambda, \text{ (has three choice)}$$

$$S \rightarrow aSb,$$

$$S \rightarrow bSa, \rightarrow \text{parallel, not subsequential.}$$

Take $\Sigma = \{a, b\}$, and let $n_a(w)$ and $n_b(w)$ denote the number of a's and b's in the string w

$$L = \{w: n_a(w) = n_b(w)\} \rightarrow \text{target language}$$

(观察得到. 需证明).

Does this grammar indeed generate the language?

Proof by induction!!

Assume that all $w \in L$ with $|w| \leq 2n$ can be derived with G

For $n = 1$, trivial

$$n=1, |w| \leq 2$$

$$\begin{aligned} S &\rightarrow aSb \Rightarrow S \rightarrow ab \\ S &\rightarrow \lambda \end{aligned}$$

$$\begin{aligned} S &\rightarrow bSa \\ S &\rightarrow \lambda \end{aligned} \Rightarrow S \rightarrow ba \checkmark$$

$$\text{思路: 证 } L(G) = L \Rightarrow \begin{cases} L(G) \subseteq L \checkmark \text{ 直接} \\ L \subseteq L(G) \end{cases}$$

Example 1.13

$$G = (V, T, S, P)$$

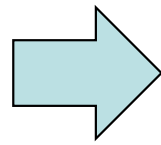
1

$$S \rightarrow SS,$$

$$S \rightarrow \lambda,$$

$$S \rightarrow aSb,$$

$$S \rightarrow bSa,$$



Take $\Sigma = \{a, b\}$, and let $n_a(w)$ and $n_b(w)$ denote the number of a's and b's in the string w

$$L = \{w: n_a(w) = n_b(w)\}$$

★ Assume that all $w \in L$ with $|w| \leq 2n$ can be derived with G

Take any $w \in L$ of length $2n+2$.

↑ 符合. $\Rightarrow w$ 以 start S 推之.

If $w = \underline{aw_1b}$, then w_1 is in L , and $|w_1| = 2n$. By assumption,

$$S \xRightarrow{*} w_1$$

Situation 1: 首尾字符

Then

$$S \Rightarrow aSb \Rightarrow aw_1b = w \quad (\text{so is } bSa)$$

Else

$$S \Rightarrow SS \Rightarrow w_1S \Rightarrow w_1w_2 = w \quad 2: \text{同.}$$

为什么一定能这样切? 设 $d = \text{Number of } a \text{ (Have read)} - Nb \text{ (Hr)}$

Equivalent of Grammars ⇒ 中间定有 $d=0$ 时刻.

每读一个字符, $d \pm 1$ 假设 a 开头. Start: $d=1$. End: $d=-1$

同理 b .

- Two grammars G_1 and G_2 are **equivalent** if they generate the same language
($L(G_1) = L(G_2)$)

- Example 1.14
- $G_1 = (\{S\}, \{a, b\}, S, P_1)$ with P_1 :
 $S \rightarrow aSb, S \rightarrow \lambda$
- $G_2 = (\{S, A\}, \{a, b\}, S, P_2)$ with P_2 :
 $S \rightarrow aAb \mid \lambda, A \rightarrow aAb \mid \lambda$

↓ 外的意思.

Pf: $S \stackrel{\alpha}{\Rightarrow} a^n S b^n$

$S \rightarrow \lambda$ must use for getting strings only contain a & b

$L(G_1) = \{a^n b^n : n \geq 0\}.$

✓ 项 → 要求.

E.g. (For quiz review)

Find grammars for $\Sigma = \{a, b\}$ that generate the sets of

(a). ** all strings with exactly 2 a's

↳ 拼接

$$S \rightarrow AaAaA \quad A \rightarrow bA/\lambda$$

↳ concatenator

(Answer Not Unique).

(b). all strings with at least 2 a's

$$S = AaAaA \quad A = bA/\lambda/aA$$

Summary:

① 一条规则只涉及一个 concatenator + alphabet.

(c) ~ no more than 3 a's

↳ more than 3

$$S = AA/AaA/AaAaA/AaAaAaA/AaAaAaAaA$$

$$A = bA/\lambda$$

② 字母前/后必加 Concatenator

③ Concatenator 必有 λ .

④. no more than 要列出 0 至 $\leq n$ 所有 situations

(d) at least 3 a's

$$S = AaAaAaA$$

$$A = \lambda \mid bA \mid aA$$

(e) start with a and end with b

$$S = aAb$$

$$A = \lambda \mid aA \mid bA$$

* For f:

不需知共有几个b. 只知b
odd/even.

S: 已全部有偶个b

A: 奇

(f) ** all strings with an even number of b's

$$S = AA \mid aSb \mid$$

A

one possible version:

$$S \rightarrow aS \mid bA \mid \lambda$$

$$A \rightarrow aA \mid bS$$

$$S \rightarrow \begin{cases} \text{全a. 变偶. } aS \\ \text{全b. 变奇. } bA \end{cases}$$

$$A \rightarrow \begin{cases} \text{全a. 变奇. } aA \\ \text{全b. 变偶. } bS \end{cases}$$

even b: 只有S可消灭